Learntify Study Plans REST API

Final project proposal

Author Juan Pablo Bovio Vallejo

Purpose Present a two-week REST API project that satisfies the course CRUD rubric and adds a useful feature to Learntify.

Proposal in one minute

I propose a Study Plans API for Learntify, a free, open-source JavaScript learning platform. A learner will be able to create a study plan, list their plans, retrieve one plan, replace its editable fields, and delete it. Every operation will use a database and require an authenticated account.

The coursework deliverable is a new, bounded module in an existing project. Learntify already has a TypeScript API, Cloudflare D1 persistence, session authentication, repository classes, and automated checks. The five Study Plans operations described here do not exist yet. This proposal defines what I will implement and demonstrate over the next two weeks.

Problem and benefit

Lessons and practice exercises help learners understand code, but learners also need a way to organize their goals. A study plan records what they intend to study, the weekly time they can commit, and whether the plan is a draft, active or completed. This gives CRUD a real purpose inside the product.

Scope for grading

• One private resource: StudyPlan. Five HTTP operations in one modular API, not five independently deployed microservices.

• English identifiers, database persistence, resource-oriented URLs, explicit OOP responsibilities, OpenAPI YAML, diagrams and repeatable HTTP-status tests.

• Existing account authentication will be reused. Public deployment, forum/chat, automated learning recommendations and the entire Learntify platform are outside this two-week delivery.

This is a proposal for tomorrow, not a claim that the CRUD module is complete. I will disclose the existing foundation and identify the newly implemented coursework clearly. Instructor acceptance of reusing an existing project remains a course-policy question.

02 Course requirements and API surface

Course requirement	Planned evidence
Five CRUD operations	GET collection; GET item; POST collection; PUT item; DELETE item.
English code and OOP	StudyPlan entity, StudyPlanService, StudyPlanRepository interface, D1StudyPlanRepository and StudyPlanController.
Read data from a database	D1 table, prepared SQL statements, persisted create/update/delete and owner-filtered reads.
Resource oriented design	Plural noun URLs under /api/v1/study-plans; HTTP verbs express the action.
YAML and diagrams	Companion OpenAPI 3.0.3 YAML; architecture, class, entity relationship and sequence diagrams.
HTTP status handling	Explicit success/error mapping plus tests for every required response listed below.

Operation and route	Required responses
GET /api/v1/study-plans	200 / 400 / 401 / 500
GET /api/v1/study-plans/{id}	200 / 400 / 401 / 404 / 500
POST /api/v1/study-plans	201 / 400 / 401 / 409 / 500
PUT /api/v1/study-plans/{id}	200 / 400 / 401 / 404 / 500
DELETE /api/v1/study-plans/{id}	204 / 400 / 401 / 404 / 500

Contract decisions

POST creates a server-generated UUID and returns the created representation plus a Location header. PUT replaces all editable fields on an existing item; it is not an upsert. DELETE returns an empty body on success. Deleting the same ID again returns 404; the final database state remains deleted.

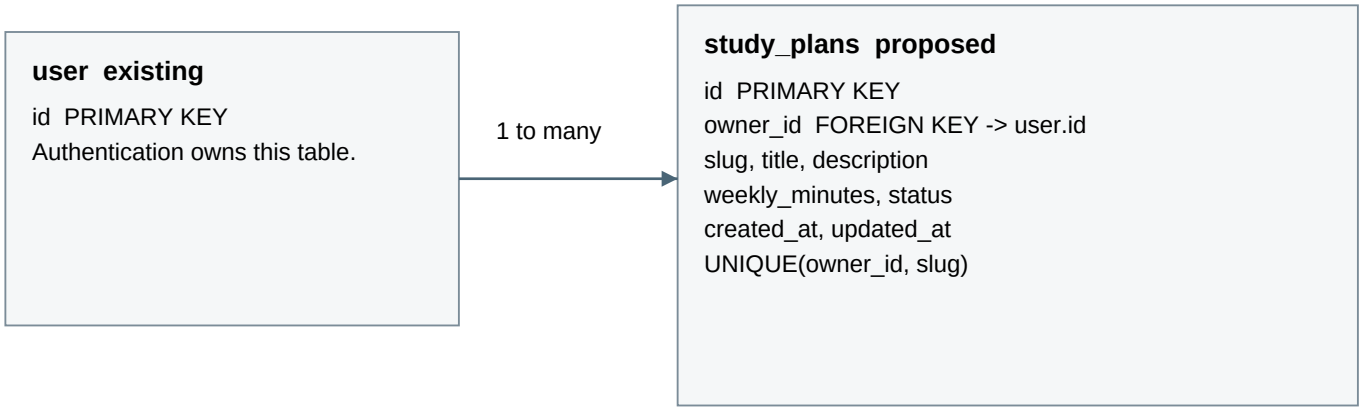
GET list returns 200 with an empty array when there are no plans. Invalid IDs, malformed JSON, invalid fields and invalid pagination return 400 in this module. Existing Learntify endpoints sometimes use 422; those older contracts will not be silently changed.

The required codes are a minimum. Existing origin and body-size protections will additionally return 403 and 413 where appropriate. Those responses are also documented in the YAML.

03 Resource model and database design

A plan belongs to one authenticated user. The server obtains ownerId from the session; clients cannot choose it. IDs and timestamps are generated by the server. UUIDs identify records; an immutable slug provides a meaningful, per-user duplicate rule.

Field	Rule
id / ownerId	Server UUID / authenticated user ID. Both immutable.
slug	3-60 lowercase letters, digits or internal hyphens. Unique within one owner. Immutable after POST.
title / description	Title: 1-120 characters after trimming. Description: 0-2000 characters.
weeklyMinutes	Integer from 15 to 1200. A planning target, not a measured activity count.
status	draft, active or completed. POST starts in draft; PUT supplies the desired state. Completion is self-reported.
createdAt / updatedAt	UTC ISO 8601 strings in JSON, stored as integer epoch milliseconds in D1.



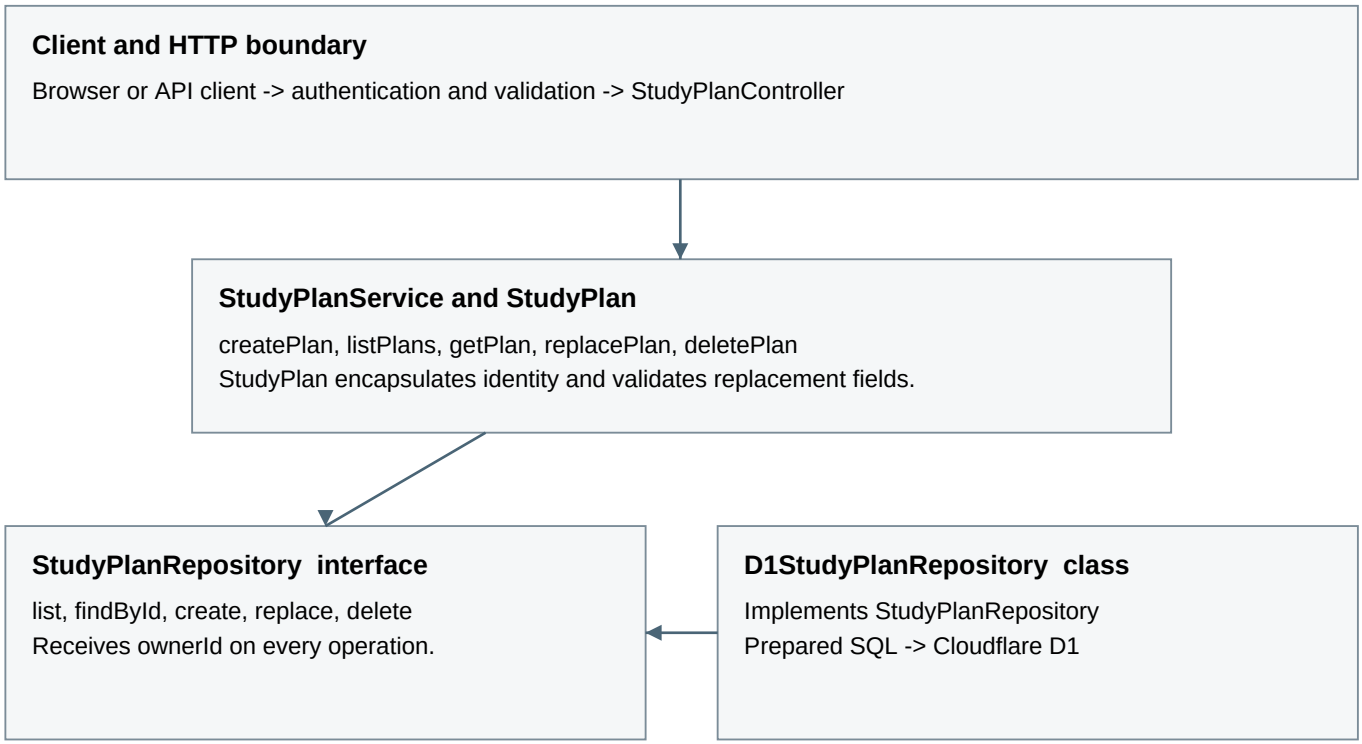
Persistence and consistency

Add a new migration without altering historical migrations. The proposed-schema.sql companion is a design artifact, not an applied migration. Every SELECT, UPDATE and DELETE includes owner_id. A database UNIQUE constraint handles concurrent duplicate POSTs atomically and maps only that known constraint to 409.

Use prepared statements and bound values. The default list uses page=1 (maximum 10000) and limit=20 (maximum 100), with optional status filtering and stable ordering by created_at DESC, id DESC. PUT uses last-write-wins for this single-user coursework scope; a future shared editor could add version checks.

04 Architecture and object oriented design

Keep one modular server. HTTP parsing belongs to the controller, business rules to the service/entity, and SQL to the D1 adapter. The domain depends on a repository interface, not Hono or Cloudflare types. Dependencies are injected at the composition root.



OOP principles in concrete terms

• Encapsulation: private entity state and controlled replacement prevent changing immutable ownerId, slug or ID through a request.

• Abstraction and polymorphism: StudyPlanService accepts StudyPlanRepository. D1StudyPlanRepository and an in-memory test implementation satisfy the same interface.

• Single responsibility: the controller maps HTTP; the service coordinates use cases; the entity enforces invariants; the adapter persists data.

• Dependency inversion: core rules refer to the repository interface. The server creates the D1 implementation and injects it. Inheritance is not required to demonstrate OOP.

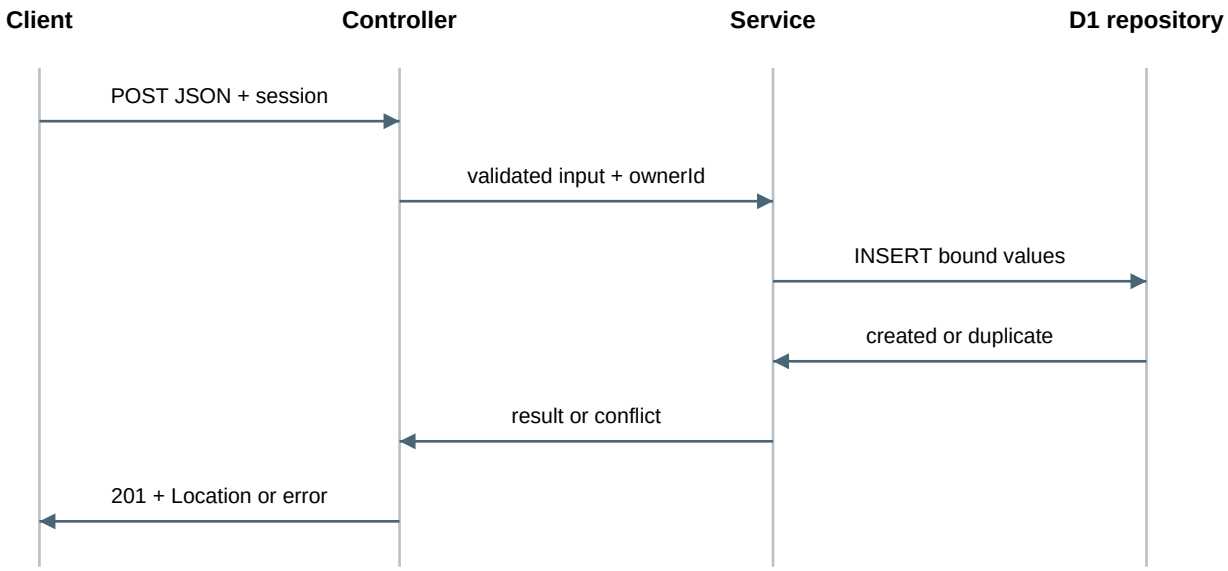
Proposed locations

```
modules/study-plans/src/        entity, service, repository interface
adapters/study-plans/src/       D1 implementation
services/server/src/study-plans/ controller and route registration
packages/api-contracts/src/     request and response schemas
services/server/migrations/     next numbered SQL migration
```

These module names are proposed additions. Existing D1PracticeRepository already demonstrates a similar adapter pattern; the coursework adds a more explicit entity/service/controller slice.

05 Request flow and access control

Example create request



The controller first authenticates the request. For writes, it verifies the trusted application Origin, enforces a bounded request body, parses JSON and validates the schema. It passes only validated fields and the session-derived ownerId to the service. Unknown input fields are rejected.

Access and errors

• No session or an expired session: 401. A valid ID that is absent or owned by another user: 404, avoiding disclosure of someone else's data.

• A duplicate immutable slug for the same owner: POST returns 409. Different owners may use the same slug. PUT cannot change the slug, so it does not introduce that conflict.

• Unexpected database or application failure: 500 with a generic message and requestId. Log useful diagnostics without session cookies, credentials or private body contents.

• Cookie authentication reuses Better Auth. Local clients sign in through the local email flow and retain the cookie. Write requests include the configured Origin. No replacement JWT system is necessary.

Authentication is evaluated before field validation for otherwise processable requests. For grading error cases, use an authenticated session and a valid Origin unless the case is specifically testing authentication or origin rejection. Infrastructure may reject an oversized request before application validation.

06 Request examples and YAML contract

Create and replace

```
POST /api/v1/study-plans
Content-Type: application/json
Origin: http://localhost:5174

{
  "slug": "javascript-foundations",
  "title": "JavaScript foundations",
  "description": "Practice functions and arrays",
  "weeklyMinutes": 180
}
```

With a valid local session, the response is 201 Created. Its Location is /api/v1/study-plans/{generated-id}. The JSON representation includes status="draft", immutable identity fields and server timestamps.

```
PUT /api/v1/study-plans/{id}

{
  "title": "JavaScript foundations and projects",
  "description": "Finish the basket calculator",
  "weeklyMinutes": 240,
  "status": "active"
}
```

PUT requires all four editable fields. It does not accept id, ownerId, slug or timestamps. Missing fields return 400; a valid but absent ID returns 404. Successful replacement returns 200 and the updated representation.

Envelope and deletion

```
{ "error": { "code": "NOT_FOUND",
             "message": "Study plan not found." },
  "meta": { "requestId": "request-identifier" } }
```

Successful JSON responses use { data, meta }. GET list also includes page and limit in meta. DELETE success is exactly 204 with zero response-body bytes; it does not return a JSON envelope.

OpenAPI deliverable

openapi-study-plans.yaml is the complete proposed OpenAPI 3.0.3 contract. It defines the five operations, cookie authentication, Origin header, request schemas, pagination, all rubric response codes, additional security errors and reusable error schemas. It documents a future module; the routes are not callable yet.

Use the local account-enabled server at http://localhost:8788 for the planned demo. The cookie scheme names the default local Better Auth session cookie. Verify the actual configured name before a deployed environment is added; secure deployments may prefix it differently.

07 Testing and acceptance evidence

Do not manufacture failures in production. Test 500 through an injected failing repository in the test environment. Verify missing resources with a valid unused UUID; malformed IDs are a separate 400 case. Use two test accounts to prove owner isolation.

Operation	Required reproducible cases
GET list	200 persisted list and empty list; 400 page=0 or invalid status; 401 no session; 500 repository throws.
GET item	200 own record; 400 malformed UUID; 401 no session; 404 missing or other owner; 500 repository throws.
POST	201 new record and Location; 400 invalid JSON/fields; 401 no session; 409 duplicate owner+slug; 500 unexpected failure.
PUT	200 complete replacement; 400 missing fields or malformed ID; 401 no session; 404 missing or other owner; 500 unexpected failure.
DELETE	204 empty body; 400 malformed ID; 401 no session; 404 missing or other owner; 500 unexpected failure.

Additional correctness tests

• Persistence: POST, read it back, restart the local Worker, and read it again. PUT and DELETE must be visible in subsequent queries.

• Concurrency: two simultaneous POSTs with one owner and slug produce one record, one 201 and one 409. Do not rely on a pre-insert existence check.

• Isolation: account B cannot list, read, update or delete account A’s plans. Confirm A’s record is unchanged.

• Boundaries: trim titles; reject unknown fields, invalid enums and out-of-range minutes. Test wrong Origin and oversized bodies separately.

• Idempotence: repeated PUT has the same editable state; repeated DELETE keeps the item absent. Request IDs and update timestamps may differ.

Definition of done

All five routes operate against D1; the listed HTTP cases pass automated tests; YAML agrees with actual responses; diagrams match the implemented classes and schema; CI is green; and a clean checkout can run the local demonstration using documented commands. Passing existing Learntify tests alone does not prove this new module is complete.

08 Two week implementation plan

Day	Deliverable
1	Present scope and existing-code disclosure. Confirm the instructor accepts five operations in one API and the chosen stack.
2	Finalize OpenAPI, request rules, entity invariants and diagrams. Prepare examples for every required status.
3	Implement StudyPlan entity and repository interface; unit-test validation and immutable fields.
4	Add the next D1 migration and D1StudyPlanRepository; test constraints, owner isolation and persisted reads.
5	Implement POST and GET list, including duplicate handling and validated pagination.
6	Implement GET by ID and PUT with full replacement and consistent 404 behavior.
7	Implement DELETE, empty 204 responses, global error mapping and origin/body-size protections.
8	Add integration tests for every required HTTP response, including injected failures.
9	Run concurrency and cross-account tests; fix defects and verify a clean local setup.
10	Reconcile YAML and diagrams with code; add schema/reference validation to CI.
11	Prepare repeatable demo requests, test accounts and local sample data with no secrets in Git.
12	Rehearse the live demo and record request/response evidence for submission.
13	Buffer for defects and instructor feedback. Avoid adding unrelated platform features.
14	Run final CI, freeze the submission revision and present the module.

This is a proposed schedule, not a promise of a known number of hours. The instructor’s deadline and available study time determine the pace. Core CRUD, correctness and documentation take priority over a new frontend.

Delivery package

Source and migration; OpenAPI YAML; diagrams; setup README; automated test results; a demo request collection or script; and the final report identifying what was newly implemented. The current proposal supplies the design artifacts first. Runtime implementation remains the two-week work.

No paid services or production deployment are needed for assessment. Use local Wrangler with local D1 and the existing local mailbox. Cloudflare staging remains a separate milestone, consistent with the current decision to defer deployment.

09 Presentation and discussion guide

Suggested opening script

My project is the Study Plans REST API, a new module for Learntify, my free JavaScript learning platform. Learntify already has authentication and a database foundation. For this course I will build a complete, owner-scoped CRUD module rather than claim the existing platform as a new submission.

A learner can create a plan, list their plans, retrieve one, replace its editable fields and delete it. The resource uses /api/v1/study-plans, with GET, POST, PUT and DELETE. It stores data in Cloudflare D1 and follows the required response codes, including 201 for creation, 409 for a duplicate slug and 204 for deletion.

The design separates a controller, service, entity and repository interface. A D1 repository implements persistence. This makes the object-oriented structure visible and lets me test database failures without breaking the real database. I will deliver the YAML contract, diagrams and tests alongside the code.

Over the next two weeks I will implement the model and persistence first, complete the routes, then test every required status and rehearse a repeatable local demonstration. The full learning platform is larger than this assignment; this submission has a clear and achievable boundary.

Questions to be ready for

• Why this resource? It is a useful learner-owned feature with natural create, read, update and delete behavior.

• Why not five microservices? The rubric lists five HTTP operations. One modular API exposes them without unnecessary distributed infrastructure; confirm the instructor’s terminology.

• Where is OOP? Entity encapsulation, a coordinating service, a controller and interchangeable repository implementations.

• Why 409 only on POST? The duplicate rule uses owner plus an immutable slug. PUT cannot alter the slug.

• Is the API finished? No. The existing foundation is working; this document proposes the coursework extension and its acceptance tests.

Evidence and references

Repository baseline inspected: services/server/src/index.ts; services/server/src/learning/practice-routes.ts; services/server/src/learning/preferences-routes.ts; adapters/practice/src/index.ts; modules/learning/src/index.ts; services/server/migrations/0006_practice_attempts.sql; .github/workflows/ci.yml.

HTTP semantics: RFC 9110, sections 9.3 and 15, https://www.rfc-editor.org/rfc/rfc9110.html. OpenAPI format and security schemes: https://spec.openapis.org/oas/v3.0.3. The assignment text supplied by the student is the authority for the required response matrix.